COMP 356
Programming Language Structures
Notes for Chapter 12 of *Concepts of Programming Languages*
Object-Oriented Programming

# 1 Object-Oriented Concepts

An object-oriented programming language must support:

- abstract data types
- inheritance
- dynamic binding of messages to methods

## 1.1 Inheritance

**Inheritance** allows one class (called the **derived class** or **subclass**) to inherit the instance variables and methods of another class (called the **parent class** or **superclass**). Use: extending or adapting an existing class to a new application without modifying the definition of the original class.

In an object-oriented programming language, a class has three "user interfaces":

- an interface that is visible to all client code (the public interface)
- an interface that is visible only to subclasses and the class itself (the protected interface)
- an interface that is visible only within the class itself (the private interface)

so subclasses are another kind of client.

A subclass can add instance variables and methods to those of its superclass(es), and can redefine (**override**) superclass methods. Overriding is useful when the superclass method is not appropriate for subclass instances (because of added instance variables etc.).

## 1.2 Dynamic Binding

The method invoked in response to a message can NOT always be determined at compile time. In object-oriented programming languages, a variable of a superclass type can contain a subclass instance. If any subclass overrides a method of the superclass, the compiler can NOT determine whether a message sent to such a variable invokes the superclass version or a subclass version of the method.

Example (Java):

```
class Shape {
  ..
  void print() {
     ..
  }
}

class Circle extends Shape {
  ..
  void print() { // overriding print from Shape
     ..
  }
}
```

```
class Square extends Shape {
  ..
  void print() { // overriding print from Shape
     ..
  }
}

class Test {
  public static void main(String argv[]) {
    Shape[] shapeList = new Shape[3];
    shapeList[0] = new Circle(); // superclass variable holding subclass instance
    shapeList[1] = new Square();
    shapeList[2] = new Shape();

    int i;
    for (i = 0; i < shapeList.length; i++) {
      shapeList[i].print(); // which print is invoked?
    }
  }
}
```

Hence, binding of messages to methods is dynamic. Significance: the above code for printing an array of shapes works correctly even if we add new subclasses of `Shape` and store instances of them in the array, as long as these new subclasses override `print()` appropriately.

Typechecking is still static - every subclass of `Shape` is guaranteed to have a `print()` method with the same number and type of parameters.

## 1.3 Subtyping

The **subtype principle**: one type is a subtype of another (called the supertype) if an instance of the subtype can always be used (correctly) where an instance of the supertype is expected. Subtypes are characterized by an "is-a" relationship: a square is a shape, an employee is a person, ...

Subclasses are often (but not always) subtypes of their superclass. For example:

- Java: a subclass can override a superclass method with a method with completely incompatible behavior.
- C++: a subclass can change the accessibility of a member (data member or member function) inherited from the superclass from `public` to `private`

If a subclass is a subtype, then existing code that manipulates a superclass instance will also manipulate a subclass instance correctly. Object-oriented programming languages generally "assume" that subclasses are subtypes, because they allow a subclass instances to be used anywhere superclass instances are expected (assigned to a variable of a superclass type, passed as a parameter to a method with a formal parameter of the superclass type, ...). Hence, defining a subclass that is not a subtype will frequently cause existing code to break. To be a subtype, a subclass must:

- inherit all instance variables and methods that are available to client code from the superclass
- not override superclass methods in incompatible ways

A subtype can add instance variables and methods.

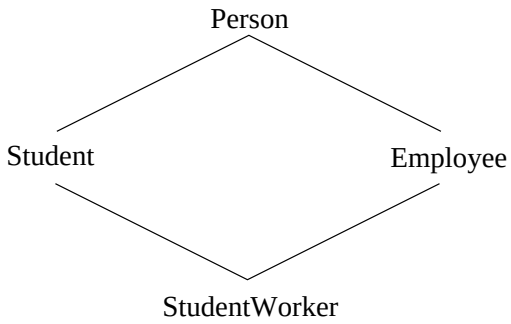
## 1.4 Multiple Inheritance

In a language with **multiple inheritance**, a subclass can have more than one (direct) parent class. Example: a cascading (pull-down) menu is-a menu, and also is-a menu item. So, it may make sense to implement class CascadingMenu as a direct subclass of Menu and of MenuItem.

One problem with multiple inheritance occurs when multiple parent classes have a member with the same name. For example, both Menu and MenuItem could have a display method with the same signature. Issues:

- which version does CascadingMenu inherit?
- what if CascadingMenu needs to use both of the parent display methods?

Another common problem occurs if both of the parent classes have a common superclass. For example:




Should class StudentWorker inherit the instance variables of class Person (name, date of birth, ...) twice? Because of the shape of the inheritance graph, this situation is called **diamond inheritance**.

Because of these difficulties, many object oriented programming languages do not allow multiple inheritance.

# 2 Object-Oriented Programming in Java

General design:

- the type system includes both objects and primitive types
- all subprograms must be defined within a class
- objects are always heap-dynamic, and are always manipulated by reference
- the programmer is not responsible for deallocating objects
- only single inheritance is allowed for classes - all classes have exactly one parent class (except `Object`)
- binding of messages to methods is dynamic, except for `final`, `private` and `static` methods
- a subclass always inherits all members of its superclass (but can override superclass methods and shadow superclass variables)
- a class can be declared `final`, which prevents any other class from using it as a parent class

Note that a class can implement multiple interfaces, and an interface can extend multiple interfaces. This is less problematic than multiple inheritance of classes because:

- variables in interfaces must be static and final (constants)
- a compiler error results if an interface inherits two constants with the same name
- a class that implements multiple interfaces containing the same message need only define one method for that message

Java also includes abstract classes, which are classes that can't have instances. Abstract classes have one or more abstract methods, which consist only of a message (no implementation). Any subclass of an abstract class that will have instances must override all abstract methods that it inherits. This is useful at "high" levels in an inheritance hierarchy - an abstract class can dictate that all subclasses have a particular method, even though it can't provide an implementation.

Example:

```
public abstract class Polygon {
// purpose: base class for (regular) shapes to inherit from
  private int n; // number of sides
  private int s; // length of a side

  public int getNumSides() {return n;}
  public int getSideLength() {return s;}
  public Polygon(int n1, int s1) {n = n1; s = s1;}
  public int perimeter() {return n * s;};

// note that this an abstract class -- no instances may be created
// enforce that any nonabstract subclass have a valid area method
  public abstract double area();
}

public final class Square extends Polygon {
// purpose: implement squares by inheriting from Polygon
// making the class final prevents any subclassing

  public Square(int sideLngth) {super(4, sideLngth);}

  public double area() { return (double) (getSideLength() * getSideLength()); }
}

public final class Triangle extends Polygon {
// purpose: implement equilateral triangles by inheriting from Polygon

  public Triangle(int sideLngth) {super(3, sideLngth);}

  public double area() { return (Math.sqrt(.75) * getSideLength()
                                 * getSideLength()); }
}
```

Note that in an array of base type `Polygon`, each element would have a valid `area()` method, i.e.:

```
Polygon [] shapeArray = new Polygon[5];
// initialize with shapes
double totArea = 0;
int i;

for (i = 0; i < shapeArray.length; i++) {
  totArea += shapeArray[i].area();
}
```

The primary difference between an interface and an abstract class is that an abstract class can define some methods for subclasses to inherit.

# 3 Object-Oriented Programming in C++

General design:

- the type system includes both objects and primitive types
- functions can be defined outside of any class

- objects can be either heap-dynamic or stack-dynamic
- the programmer is responsible for deallocating objects
- multiple inheritance is allowed - a class can have 0, 1 or many parent classes
- binding of member function calls to member functions can be either static or dynamic

## 3.1 Inheritance in C++

Single inheritance is specified by:

`class` *subclass* `:` *access superclass*

where *access* is either `public` or `private`. For example:

```
class Employee : public Person {
 ...
```

If the access is `public`, then the public and protected members of the superclass become public and protected members of the subclass. If the access is `private`, then the public and protected members of the superclass become private members of the subclass. Hence, private subclasses are almost never subtypes!

Private subclasses are used to inherit the functionality (but not the interface) of a superclass. Example (from the text): `Queue` could be a private subclass of `List`, but not all list operations should be applied to queues. A private subclass can selectively export some of its inherited members as public or protected, so operations that are appropriate can be made available to client code.

Design alternative: use composition rather than private inheritance. For example, `Queue` can be implemented using a data member that is a `List`.

Multiple inheritance:

- is specified using a comma separated list of parent classes
- in diamond inheritance, members of the common base class are inherited only once if inheritance is specified as virtual
- otherwise, all members of all parent classes are inherited
- members with the same name are distinguished using scope resolution

Example:

```
#include <iostream>

class Person {
  private:
  std::string name;
  int yearOfBirth;

  public:
    Person(std::string n, int y) { name = n; yearOfBirth = y;}
    virtual void vprint() {std::cout << name << " was born in " << yearOfBirth;}
};

class Employee : public virtual Person {
  private:
    double salary;
    int idNum;
```

```
  public:
    Employee(std::string n, int y, double s) : Person(n, y) {
      salary = s;
    }

    virtual void vprint() {
      Person::vprint(); // must use scope resolution to avoid recursive call
      std::cout << " and makes $" << salary;
    }
};

class Student : public virtual Person {
  private:
    double gpa;
    int idNum;

  public:
    Student(std::string n, int y, double g) : Person(n, y) {
      gpa = g;
    }

    virtual void vprint() {
      Person::vprint();
      std::cout << " and has a GPA of " << gpa;
    }
};

class StudentWorker : public Student, public Employee {
// a StudentWorker has one name and year of birth
  public:
    StudentWorker(std::string n, int y, double s, double g) :
       Person(n, y), Student(n, y, g), Employee(n, y, s) { }

    virtual void vprint() {
      Student::vprint();
      std::cout << std::endl;
      Employee::vprint();
    }
};


int main() {
  Person bill("bill", 1987);
  bill.vprint();
  std::cout << std::endl;
  Employee ellen("ellen", 1975, 65000);
  ellen.vprint();
  std::cout << std::endl;
  Student jane("jane", 1993, 3.75);
  jane.vprint();
  std::cout << std::endl;
  StudentWorker ed("ed", 1995, 10000, 3.66);
  ed.vprint();
  std::cout << std::endl;
```

```
}
```

Notes:

- class `StudentWorker` inherits `name` and `yearOfBirth` only once
- class `StudentWorker` inherits `salary` and `vprint` from `Employee`
- class `StudentWorker` inherits `gpa` and `vprint` from `Student`
- within `StudentWorker`, scope resolution is used to specify which inherited `vprint` is meant

## 3.2 Dynamic Binding

Situation: a variable of a superclass type is assigned a subclass object. The variable is then used to invoke an overridden method.

- if method binding is dynamic, the subclass version of the method is invoked
- if method binding is static, the superclass version of the method is invoked

Note that in static binding, the method that is invoked matches the declared type of the variable, so the compiler can determine which method to invoke. In dynamic binding, there is no way to determine which method to invoke until runtime.

In C++:

- messages are statically bound to member functions unless the member function is explicitly declared `virtual`
- if the object used to invoke the method is stack-dynamic, binding is also static (even if the method is virtual)

Static binding is more efficient, so C++ gives programmers the option to only use dynamic binding when it is needed.

Example:

```
#include <iostream>

class Person {
  private:
  std::string name;
  int yearOfBirth;

  public:
    Person(std::string n, int y) { name = n; yearOfBirth = y;}
    virtual void vprint() {std::cout << name << " was born in " << yearOfBirth;}
    void sprint() {std::cout << "in Person" << std::endl;}
};

class Employee : public Person {
  private:
    double salary;

  public:
    Employee(std::string n, int y, double s) : Person(n, y) {
      salary = s;
    }

    virtual void vprint() {
```

```
      Person::vprint();
      std::cout << " and makes $" << salary;
    }
    void sprint() {std::cout << "in Employee" << std::endl;}
};

int main() {
  Person *ellen = new Employee("ellen", 1975, 65000); // heap-dynamic instance
  ellen->vprint(); // invokes Employee::vprint
  std::cout << std::endl;
  ellen->sprint(); // invokes Person::sprint

  Person bill("bill", 1987); // stack-dynamic instance
  Employee ed("ed", 1995, 10000);
  bill = ed;
  bill.vprint(); // invokes Person::vprint
  std::cout << std::endl;
  bill.sprint(); // invokes Person::sprint
}
```

Stack-dynamic instances are stored directly in a variable, so their size in memory is fixed. When a subclass instance is assigned to such a variable, it is "truncated" (all data members added by the subclass are removed) so that it will fit. Hence, subclass methods can NOT be invoked using such a variable.

Heap-dynamic instances are referred to by pointers (or references). A pointer to a subclass instance can be assigned to a variable of type pointer to superclass without any loss of information. Hence, C++ programs that use virtual methods almost always manipulate pointers to objects, rather than objects directly. A variable of type pointer to superclass is sometimes called a **polymorphic variable**, since it can hold values of different types.

# 4 Implementation of Objects, Classes and Methods

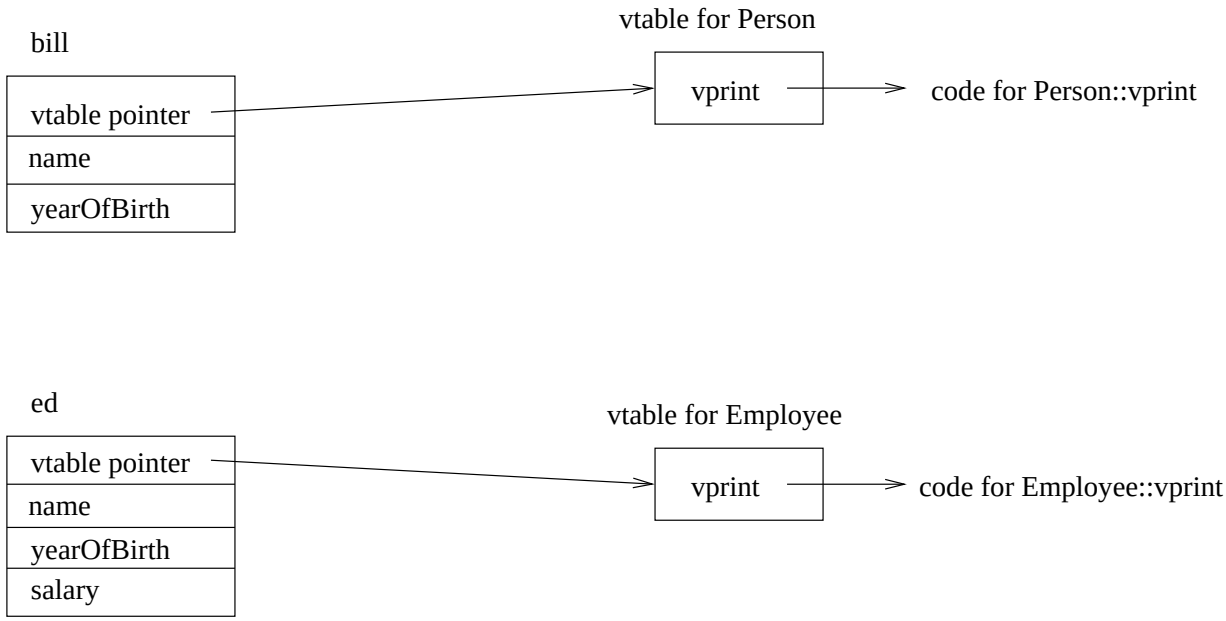
Conceptually, each object encapsulates its own state (instance variables) and operations (methods). However, the state is different for each object, while the operations are the same for all instances of the same class.

Instance variables (data members) are typically organized as a record. Since the structure is static for each class, instances variables can be accessed using fixed offsets from the beginning of such a record. Subclasses simply copy the record for the parent class, and then add an addition field for each of their new instance variables.

For statically bound methods, no special implementation support is needed - the method to invoke can be determined at compile time.

For classes that have dynamically bound methods (i.e. a C++ class with at least one virtual method, a Java class with at least one (nonstatic) method), a table containing pointers to all dynamically bound methods is constructed. This table is called a **virtual method table** or **vtable**. Each instance of such a class contains a pointer to the vtable for the class. When the instance is used to invoke a method, the vtable is consulted to determine which "version" to invoke.

Consider the previous example of classes `Person` and `Employee`, with `ed` as an instance of class `Employee` and `bill` as an instance of class `Person`:




Notes:

- For the following code:

```
Person *p = new Employee("edna", 1945, 87500);
p->vprint();
```

`Employee::vprint` is invoked, because the vtable pointer is set when the `Employee` object is created, and assigning a pointer does not require any truncation.

- when a stack-dynamic instance is assigned to, the vtable pointer within the instance is not changed. For example:

```
Person bill("bill", 1987);
Employee ellen("ellen", 1975, 65000);
bill = ellen;
bill.vprint();
```

`Person::vprint` is invoked. The `name` and `yearOfBirth` are copied from `ellen` to `bill`, but `bill`'s vtable pointer still must point to the vtable for `Person` because using `Employee::vprint` requires a `salary` data member.

- multiple inheritance complicates the use of vtables. Standard implementation:
  - construct a vtable for the subclass and the first parent class as usual in single inheritance
  - for each additional parent class, construct a vtable consisting of the pointers to the virtual methods inherited by the subclass
  - each instance of the subclass contains a vtable pointer to each of the vtables constructed in the previous steps